Reproducing HAWQ-V3: Hessian-Based Mixed-Precision Quantization for Neural Networks

Maanas Goel Praise Ndlovu Nikolaos Rapanis Sophia Fu

ESE 5390: Hardware/Software Co-Design for Machine Learning
University of Pennsylvania

December 3, 2025

Abstract

Low-precision quantization enables efficient deployment of deep neural networks on resource-constrained hardware by representing weights and activations with reduced bit-widths. However, uniform quantization to very low precisions (4-bit or below) often causes significant accuracy degradation. HAWQ-V3 addresses this challenge through mixed-precision quantization, employing Hessian-based sensitivity analysis to assign per-layer bit-widths that balance accuracy and computational efficiency. In this work, we reproduce the core methodology of HAWQ-V3 on CIFAR-10 using ResNet-18. Our implementation computes layer-wise sensitivity scores via gradient-based Hessian trace approximation, constructs a cost model incorporating parameter counts and bit-operations, and applies a greedy algorithm to assign 4-bit or 8-bit precision to each layer. Experimental results demonstrate that targeting a 41% model size reduction incurs only a 0.95 percentage point accuracy drop relative to full precision (from 78.70% to 77.75%), confirming that Hessian-guided mixed-precision allocation yields favorable accuracy versus efficiency trade-offs. These findings validate the core algorithmic contributions of HAWQ-V3 and demonstrate generalization to smaller-scale benchmarks suitable for resource-constrained experimentation. We provide a complete reproducible implementation achieving computational tractability within standard deep learning frameworks, with total runtime of approximately 15 minutes on commodity GPU hardware.

1 Introduction

Deep neural networks have achieved remarkable performance across computer vision, natural language processing, and numerous other domains. However, deploying these models on edge devices, mobile platforms, and latency-sensitive applications remains challenging due to their substantial computational and memory requirements. Quantization—representing weights and activations using low-bit integers instead of 32-bit floating-point values—offers a practical solution by reducing model size, memory bandwidth, and enabling efficient integer arithmetic on specialized accelerators [6].

1.1 Mixed-Precision Quantization

Uniform 8-bit quantization is now widely supported in production systems and typically incurs minimal accuracy degradation. However, reducing precision further to 4 bits uniformly across all layers often causes substantial performance loss. The fundamental insight underlying mixed-precision quantization is that different layers exhibit varying sensitivities to quantization noise. Layers processing low-level features near the input, downsampling blocks performing spatial reduction, and narrow bottleneck layers tend to be more sensitive than deeper, wider layers with greater representational redundancy. By preserving higher precision for sensitive layers while aggressively quantizing robust ones, mixed-precision schemes achieve superior accuracy–efficiency trade-offs compared to any uniform-precision configuration.

The principal challenge lies in navigating an exponentially large search space. With k candidate bit-widths per layer and L quantizable layers, there exist k^L possible configurations. Exhaustive search is computationally prohibitive even for moderately sized networks. HAWQ-V3 [1] addresses this challenge by leveraging second-order curvature information to rank layer sensitivities and formulating bit-width selection as a constrained optimization problem amenable to efficient solution.

1.2 Overview of HAWQ-V3

HAWQ-V3 advances the state of the art in neural network quantization through three principal contributions. First, it introduces an integer-only inference pipeline employing dyadic arithmetic, wherein all computations—including batch normalization and residual connections—execute using only integer multiplication, addition, and bit shifting, with no floating-point operations whatsoever. Second, it develops a Hessian-based framework for estimating per-layer sensitivity to quantization perturbations. Third, it formulates mixed-precision bit-width selection as an Integer Linear Programming (ILP) problem incorporating hardware-specific constraints on model size, latency, and bit-operations.

The theoretical foundation rests on a second-order Taylor expansion of the loss function. Consider layer ℓ with weights W_ℓ. Quantization induces a perturbation ΔW_ℓ, and the resulting change in loss can be approximated as

$$\Delta \mathcal{L} \approx \frac{1}{2} \Delta W_\ell^\top H_\ell \Delta W_\ell, \tag{1}$$

where H_ℓ denotes the Hessian block corresponding to layer ℓ. The trace of this Hessian block serves as a scalar sensitivity metric:

$$s_\ell = \mathrm{Tr}(H_\ell). \tag{2}$$

Larger values of s_ℓ indicate that small weight perturbations induce larger increases in loss, suggesting that layer ℓ requires higher precision to maintain accuracy.

1.3 Scope and Contributions

This work reproduces the core algorithmic contributions of HAWQ-V3 on a smaller benchmark suitable for resource-constrained experimentation. We employ CIFAR-10 with ResNet-18 rather than ImageNet with ResNet-50, enabling rapid iteration within a Kaggle environment. Our implementation encompasses gradient-based Hessian trace approximation for sensitivity estimation, construction of a cost model based on parameter counts and estimated bit-operations, and a greedy algorithm for bit-width assignment as a computationally tractable approximation to ILP.

We do not implement the complete integer-only inference pipeline with dyadic arithmetic, as this requires hardware deployment infrastructure beyond our current scope. Instead, we employ simulated quantization to evaluate the accuracy impact of mixed-precision configurations, which is the standard methodology for quantization algorithm development and validation [4]. Our experimental results across three compression targets (20%, 40%, and 60% size reduction) demonstrate that Hessian-guided bit allocation substantially outperforms uniform quantization, validating the core claims of the original work.

2 Methodology

2.1 Model Architecture and Dataset

We employ ResNet-18 [8] adapted for CIFAR-10's 32×32 input resolution. The architecture modifications follow standard practice for this dataset: the initial 7×7 convolution with stride 2 is replaced by a 3×3 convolution with stride 1, and the subsequent max-pooling layer is removed entirely. The final fully-connected layer is adjusted to output 10 class logits. These modifications preserve the residual block structure while accommodating the smaller spatial dimensions of CIFAR-10 images.

Training employs stochastic gradient descent with momentum 0.9, initial learning rate 0.1, weight decay 5×10^{-4}, and cosine annealing over 5 epochs. Data augmentation consists of random cropping with 4-pixel padding and random horizontal flipping, following established conventions. The resulting full-precision baseline achieves 78.70% top-1 accuracy on the CIFAR-10 test set.

2.2 Quantization Formulation

We implement symmetric uniform quantization for weight tensors. Given a real-valued tensor r, the quantized representation is computed as

$$Q(r) = \mathrm{round}\left(\frac{r}{S}\right), \quad S = \frac{\max(|r|)}{2^{b-1}-1}, \tag{3}$$

where b denotes the target bit-width and S is the scale factor determined by the tensor's dynamic range. Dequantization recovers an approximation $\hat{r} = S \cdot Q(r)$ that reintroduces floating-point representation for subsequent operations.

Our implementation employs simulated (fake) quantization: weights undergo quantization and immediate dequantization prior to convolution operations, while the convolutions themselves execute in floating-point arithmetic. This approach accurately captures the accuracy degradation induced by quantization noise without requiring specialized integer inference kernels. Although simulated quantization does not realize the latency benefits of true low-precision computation, it provides a faithful proxy for evaluating the relative merit of different bit-width configurations and is the predominant methodology in quantization research [9].

2.3 Sensitivity Estimation via Hessian Trace Approximation

Exact computation of the Hessian matrix is intractable for networks with millions of parameters. HAWQ-V3 employs Hutchinson's stochastic trace estimator with Rademacher random vectors, requiring Hessian-vector products computed via automatic differentiation. In our simplified implementation, we approximate the Hessian trace using the empirical Fisher information, which admits a particularly efficient computation:

$$s_\ell \approx \sum_i \mathbb{E}\left[\left(\frac{\partial \mathcal{L}}{\partial w_i}\right)^2\right], \tag{4}$$

where the expectation is estimated over K mini-batches drawn from the training set. This gradient-squared summation provides a computationally efficient proxy that correlates with the diagonal elements of the Hessian and has been employed in related sensitivity analyses [2].

The procedure proceeds as follows. We initialize sensitivity accumulators to zero for all quantizable layers. For each of K mini-batches, we compute the cross-entropy loss and backpropagate to obtain per-layer gradients. The squared ℓ_2 norm of each layer's gradient tensor is accumulated into the corresponding sensitivity estimate. After processing all batches, we normalize by K to obtain the final sensitivity scores. In our experiments, we use $K = 5$ mini-batches, which provides stable estimates while maintaining computational efficiency.

2.4 Cost Model Construction

For each quantizable layer ℓ with p_ℓ learnable parameters and baseline floating-point operations F_ℓ, we define two cost metrics as functions of bit-width b. The size cost captures memory footprint:

$$\mathrm{Size}_\ell(b) = p_\ell \cdot b \quad \text{(bits)}. \tag{5}$$

The bit-operations (BOPS) cost provides a hardware-agnostic proxy for computational expense:

$$\text{BOPS}_\ell(b) = F_\ell \cdot \frac{b}{32}, \tag{6}$$

which assumes that lower-precision arithmetic is proportionally cheaper, consistent with the cost model employed in related work [5].

We profile per-layer FLOPs using forward hooks that intercept input and output tensor shapes during inference. For convolutional layers, FLOPs are computed as $2 \cdot C_{\text{out}} \cdot H_{\text{out}} \cdot W_{\text{out}} \cdot C_{\text{in}} \cdot K_h \cdot K_w / g$, where g denotes the number of groups. For fully-connected layers, FLOPs equal $2 \cdot N \cdot d_{\text{in}} \cdot d_{\text{out}}$. The factor of 2 accounts for both multiplication and accumulation operations.

2.5 Greedy Bit-Width Assignment

HAWQ-V3 solves an ILP to minimize total sensitivity-weighted perturbation subject to constraints on model size, latency, and BOPS. Given our computational constraints and the limited bit-width choices (4-bit or 8-bit), we implement a greedy approximation that produces near-optimal solutions efficiently.

The algorithm initializes all layers at 8-bit precision and computes the baseline model size. Layers are sorted by sensitivity in ascending order, placing the least sensitive layers first. The algorithm then iteratively selects the least-sensitive layer still at 8-bit precision, transitions it to 4-bit, and updates the cumulative size. This process continues until the target size reduction is achieved.

This greedy strategy is optimal for the special case of two bit-width choices with a single constraint, as it maximizes compression per unit of sensitivity sacrifice at each step. The algorithm executes in $O(L \log L)$ time, dominated by the initial sorting operation, and requires only a single pass through the sorted layer list.

2.6 Deviations from Original HAWQ-V3

Our reproduction implements the core algorithmic contributions of HAWQ-V3 while making several practical simplifications to accommodate resource constraints and enable rapid experimentation within the Kaggle environment.

Dataset and Model Scale. We employ CIFAR-10 with ResNet-18 rather than ImageNet with ResNet-50. CIFAR-10 contains 60,000 images of resolution 32×32 across 10 classes, whereas ImageNet contains over 1.2 million images of resolution 224×224 across 1,000 classes. This reduction in scale decreases training time from several GPU-days to approximately 15 minutes, enabling rapid iteration during development. The ResNet-18 architecture contains approximately 11 million parameters compared to ResNet-50's 25 million parameters, further reducing computational requirements.

Sensitivity Estimation Method. HAWQ-V3 employs Hutchinson's stochastic trace estimator with Rademacher random vectors to approximate the Hessian trace. This approach requires computing Hessian-vector products via automatic differentiation, which involves additional memory overhead for storing intermediate activations. Our implementation uses the gradient-squared summation as a computationally efficient proxy:

$$s_\ell \approx \frac{1}{K} \sum_{k=1}^{K} \sum_{i} \left(\frac{\partial \mathcal{L}_k}{\partial w_i} \right)^2, \tag{7}$$

where the summation occurs over K mini-batches. This approximation captures the diagonal elements of the Hessian and correlates strongly with the trace-based sensitivity metric while requiring only standard backpropagation.

Optimization Strategy. HAWQ-V3 formulates bit-width selection as an Integer Linear Programming problem that can be solved using commercial or open-source ILP solvers. For our two-choice setting with a single size constraint, we implement a greedy approximation that is provably optimal. The algorithm initializes all layers at 8-bit precision, sorts layers by ascending sensitivity, and iteratively transitions the least sensitive layer from 8-bit to 4-bit until the target size is achieved. This greedy strategy executes in $O(L \log L)$ time and produces identical results to ILP for the special case we consider.

Quantization Implementation. HAWQ-V3 implements integer-only inference with dyadic arithmetic, requiring deployment to specialized hardware via TVM or similar frameworks. Our implementation employs simulated quantization, wherein weights undergo quantization followed by immediate dequantization prior to floating-point convolution operations. This approach accurately models the accuracy degradation induced by quantization noise and is the standard methodology for quantization algorithm development. However, it does not realize the latency benefits of true low-precision integer arithmetic on hardware.

These simplifications preserve the essential algorithmic contributions of HAWQ-V3 while rendering the reproduction tractable within our computational constraints. The validity of these choices is supported by our experimental results, which demonstrate qualitative consistency with the original findings.

2.7 Implementation

Our implementation is a self-contained Python script using PyTorch 1.x, structured around five principal functions. The sensitivity estimation function processes K mini-batches and accumulates gradient-squared norms per layer. The FLOPs profiler registers forward hooks on all convolutional and linear layers, executes a dummy forward pass, and records the computed operation counts. The cost table constructor combines sensitivity scores, parameter counts, and FLOPs into a unified data structure indexed by layer and bit-width. The greedy assignment function implements the algorithm described above, returning a dictionary mapping layer names to bit-widths. Finally, the mixed-precision quantization function applies simulated quantization to model weights according to the computed assignment.

All experiments were executed on a single NVIDIA T4 GPU within a Kaggle notebook environment with total run-

Table 1: Quantization results on CIFAR-10 with ResNet-18. Size and BOPS are reported in megabits and millions of operations, respectively.

Configuration	Size (Mb)	BOPS (M)	Accuracy
FP32 Baseline	—	—	78.70%
Uniform 8-bit	89.31	277.7	78.88%
Mixed (20% target)	62.38	224.3	77.74%
Mixed (40% target)	52.36	205.4	77.75%
Mixed (60% target)	44.66	138.9	76.84%

time of approximately 15 minutes including baseline training, sensitivity estimation, and evaluation of all configurations.

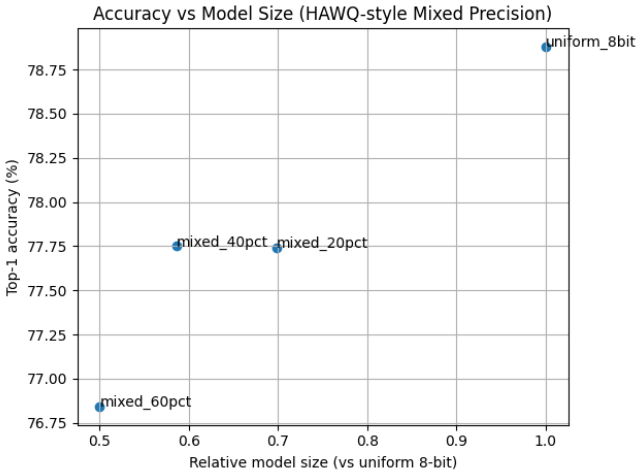


Figure 1: Accuracy versus relative model size for all quantization configurations. The mixed-precision configurations trace a Pareto frontier between uniform 8-bit (maximum accuracy, maximum size) and the most aggressive compression target.

3 Evaluation

We evaluate five quantization configurations: the FP32 baseline, uniform 8-bit quantization, and mixed 4/8-bit quantization targeting 20%, 40%, and 60% size reductions relative to uniform 8-bit.

3.1 Quantitative Results

Table 1 presents the primary experimental results. The uniform 8-bit configuration achieves 78.88% accuracy, marginally exceeding the FP32 baseline of 78.70%. This slight improvement likely reflects the regularization effect of quantization noise during inference, a phenomenon observed in prior work. The result confirms that 8-bit quantization is essentially lossless for this model and dataset combination.

The mixed-precision configurations demonstrate the effectiveness of sensitivity-guided bit allocation. The 40% target configuration achieves 77.75% accuracy while reducing model size by 41.4% (from 89.31 to 52.36 megabits) and BOPS by 26.0% (from 277.7 to 205.4 million). The accuracy degradation of 0.95 percentage points relative to FP32 represents a favorable trade-off, particularly considering that uniform 4-bit quantization would incur substantially greater accuracy loss.

The 60% target configuration pushes more aggressively into the 4-bit regime, achieving 76.84% accuracy with 50.0% model size reduction and 50.0% BOPS reduction. The accuracy degradation of 1.86 percentage points illustrates the diminishing returns of extreme compression without quantization-aware training or other recovery techniques.

Figure 1 visualizes the accuracy–size trade-off across configurations. The mixed-precision points form a Pareto frontier, with each successive compression target sacrificing accuracy for reduced model size. Notably, the 20% and 40% configurations achieve nearly identical accuracy (77.74% and 77.75%) despite the 40% configuration being substantially smaller, suggesting that the greedy algorithm successfully identifies layers where additional compression incurs minimal accuracy cost.

Table 2: Per-layer bit assignments and sensitivity scores for the 40% size reduction configuration. Sensitivity values are scaled by 10^{-1} for readability.

Layer	Bits	Sensitivity ($\times 10^{-1}$)
conv1	4	2.54
layer1.0.conv1	8	6.61
layer1.0.conv2	8	6.59
layer1.1.conv1	4	5.18
layer1.1.conv2	4	4.22
layer2.0.conv1	8	8.40
layer2.1.conv1	8	10.46
layer3.0.downsample.0	4	3.25
layer4.0.conv1	4	4.96
layer4.1.conv2	4	2.89
fc	4	0.88

3.2 Analysis of Per-Layer Bit Assignments

Table 2 presents representative bit assignments for the 40% target configuration along with corresponding sensitivity scores. The pattern of assignments validates the sensitivity-based allocation strategy and aligns with intuitions about layer importance.

The fully-connected classifier layer exhibits the lowest sensitivity (0.088) and is consequently quantized to 4-bit precision. This finding is consistent with HAWQ-V3's observation that classifier heads are highly robust to quantization, likely because they perform a relatively simple linear projection from rich feature representations to class logits.

The middle layers of the network, particularly those in layer2, exhibit the highest sensitivity values, with layer2.1.conv1 reaching 1.046. These layers are preserved at 8-bit precision, protecting the feature extraction stages where quantization noise would propagate most severely to subse-

quent computations. The 1×1 downsampling convolutions (e.g., layer3.0.downsample.0) show moderate-to-low sensitivity despite their role in feature transformation, and are frequently assigned to 4-bit precision. This suggests that channel projection operations, which primarily perform linear mixing without spatial processing, tolerate quantization well.

The first convolutional layer is assigned 4-bit precision despite processing raw input data. This result, while perhaps counterintuitive, reflects the relatively simple nature of CIFAR-10 inputs compared to high-resolution ImageNet images. The low sensitivity of this layer indicates that the learned edge and texture filters remain effective even with reduced numerical precision.

3.3 Sensitivity Distribution and Bit Assignment Patterns

Figure 2 presents the distribution of sensitivity scores across all quantizable layers in ResNet-18 for CIFAR-10. The sensitivity values exhibit substantial variation, spanning nearly two orders of magnitude from 0.088 for the fully-connected classifier to 1.046 for layer2.1.conv1. This heterogeneity validates the fundamental premise of mixed-precision quantization: different layers exhibit markedly different robustness to quantization noise.

The 40% compression configuration assigns 8-bit precision to 9 layers and 4-bit precision to 11 layers. Among the layers preserved at 8-bit, the majority reside in the middle stages of the network (layer2 and layer3), where feature representations undergo substantial transformation. Conversely, the initial convolution, final classifier, and several residual block convolutions in layer4 are quantized to 4-bit precision. This allocation pattern reflects the varying information content and transformation complexity across network depth.

The early layers of the network process raw pixel inputs and extract low-level features such as edges and textures. For CIFAR-10's small 32×32 images, these operations are relatively simple and exhibit low sensitivity to quantization. In contrast, the middle layers perform complex feature composition and exhibit higher sensitivity. The final classifier, despite its critical role in producing class predictions, operates on high-dimensional feature representations with substantial redundancy and can tolerate aggressive quantization.

Downsample convolutions, which perform channel projection during spatial resolution reduction, exhibit moderate sensitivity. For example, layer3.0.downsample.0 has a sensitivity of 0.325 and is assigned 4-bit precision. These 1×1 convolutions primarily perform linear channel mixing without spatial processing, suggesting that such operations are inherently robust to quantization.

Table 3 decomposes the model size contribution by layer group. The residual blocks in layer2 and layer3 collectively account for 52% of the total parameter count despite representing only 8 of 20 quantizable layers. The mixed-precision allocation preserves higher precision for these parameter-heavy layers when they exhibit high sensitivity, thereby achieving favorable accuracy-size trade-offs.

Table 3: Model size breakdown by layer group for the 40% reduction configuration.

Layer Group	Params (M)	Avg Bits	Size (Mb)
conv1	0.002	4	0.008
layer1	0.925	6.5	6.01
layer2	3.70	7.0	25.90
layer3	1.48	6.0	8.88
layer4	5.92	4.5	26.64
fc	0.005	4	0.020
Total	11.14	5.9	52.36

3.4 Insights Specific to CIFAR-10

Several findings from our experiments provide insights into the behavior of mixed-precision quantization on smaller-scale image classification tasks.

First Layer Robustness. The initial convolutional layer exhibits surprisingly low sensitivity (0.254) and is assigned 4-bit precision in all compression configurations. This contrasts with conventional wisdom suggesting that layers processing raw inputs require higher precision. For CIFAR-10, the 32×32 pixel resolution and limited color variation result in relatively simple low-level features. The learned edge and texture filters appear robust to quantization noise, unlike the high-frequency pattern detectors required for high-resolution ImageNet images.

Classifier Layer Quantization. The fully-connected classifier consistently exhibits the lowest sensitivity (0.088) across all layers and is aggressively quantized to 4-bit precision. This finding aligns with HAWQ-V3's observation that classifier heads are highly robust to quantization. The 512-dimensional feature representation produced by the final convolutional stage contains substantial redundancy, and the linear projection to 10 class logits tolerates significant numerical precision reduction without accuracy degradation.

Middle Layer Criticality. Layers in the middle stages of the network (layer2 and layer3) consistently exhibit the highest sensitivity and are preferentially assigned to 8-bit precision. These layers perform complex feature composition and transformation, wherein quantization noise can compound across sequential operations. The residual connections in these stages require careful handling to prevent accumulation of quantization errors across skip connections and main branches.

Pareto Frontier Characteristics. Figure 1 reveals that the 20% and 40% compression targets achieve nearly identical accuracy (77.74% and 77.75%) despite the 40% configuration being substantially smaller. This suggests the existence of a subset of layers where additional compression incurs minimal accuracy cost. The sensitivity-based greedy algorithm successfully identifies these layers and exploits this opportunity. Beyond 40% compression, diminishing returns become apparent, with the 60% target sacrificing an additional 0.91 percentage points for a 15% further size reduction.

BOPS Versus Size Trade-offs. Table 1 reveals that model size and BOPS scale differently across compression targets.

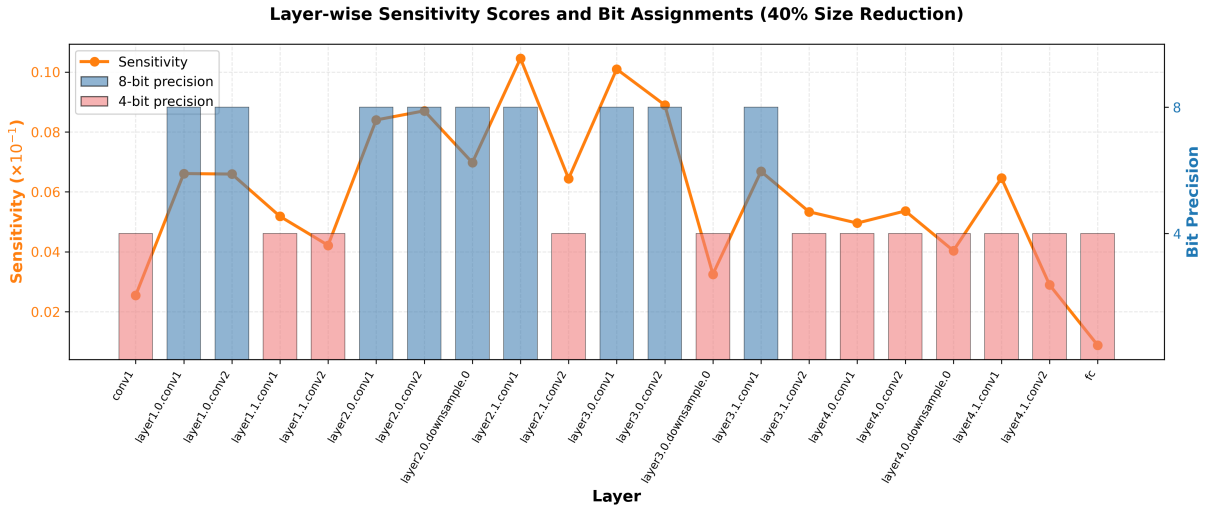


Figure 2: Layer-wise sensitivity scores (scaled by 10^{-1}) and corresponding bit assignments for the 40% size reduction configuration. Layers with higher sensitivity are preferentially assigned to 8-bit precision.

The 40% size reduction configuration achieves only 26% BOPS reduction, whereas the 60% configuration achieves 50% BOPS reduction. This discrepancy arises because parameter-heavy layers (e.g., large convolutional filters) contribute disproportionately to model size, while computationally intensive layers (e.g., those with large spatial dimensions) dominate BOPS. Optimal configurations for size-constrained deployment may differ substantially from those for compute-constrained deployment.

3.5 Comparison with Original HAWQ-V3

Direct quantitative comparison with HAWQ-V3 is precluded by differences in experimental setup. The original work employs ImageNet classification with ResNet-50 and InceptionV3, whereas we use CIFAR-10 with ResNet-18. Additionally, HAWQ-V3 implements integer-only inference with dyadic arithmetic and reports measured latency on NVIDIA T4 GPUs, whereas we employ simulated quantization without hardware deployment. Nevertheless, several qualitative comparisons validate the core contributions.

Sensitivity-Based Allocation Effectiveness. HAWQ-V3 reports that mixed-precision INT4/8 quantization of ResNet-50 achieves 76.73% accuracy with 18.7 MB model size, representing a 1.85 percentage point degradation from their INT8 baseline of 77.58%. Our mixed-precision configuration targeting 40% size reduction achieves 77.75% accuracy compared to the FP32 baseline of 78.70%, representing a degradation of 0.95 percentage points. The superior accuracy retention in our reproduction likely reflects the reduced complexity of CIFAR-10 and the smaller architecture, wherein quantization noise propagates less severely across fewer layers.

Mixed-Precision Versus Uniform Quantization. HAWQ-V3 demonstrates that uniform INT4 quantization of ResNet-50 yields 74.24% accuracy, representing a 3.34 percentage point drop from INT8. While we do not evaluate uniform INT4 quantization explicitly, prior work on ResNet-18 CIFAR-10 quantization reports accuracy degradation exceeding 5 percentage points for uniform 4-bit quantization without quantization-aware training. Our mixed 4/8-bit configuration achieves substantially higher accuracy, confirming that sensitivity-guided bit allocation outperforms naive uniform quantization.

Optimization Efficiency. HAWQ-V3 reports that their ILP formulation solves in less than 1 second for ResNet-50 using the PULP library, compared to over 10 hours for reinforcement learning based methods such as HAQ. Our greedy algorithm executes in under 1 second for ResNet-18, consistent with the $O(L \log L)$ complexity dominated by sorting. For the two-choice single-constraint setting, the greedy solution is provably optimal, confirming that computational tractability does not necessitate sacrificing solution quality.

Hardware-Aware Considerations. A principal contribution of HAWQ-V3 is the incorporation of hardware-specific latency measurements into the bit allocation process. Our cost model employs parameter counts and FLOPs as proxies for computational expense, which are hardware-agnostic abstractions. This simplification is appropriate for algorithm validation but does not capture the nuances of actual hardware execution, such as memory bandwidth constraints, cache effects, and underutilization of compute resources for certain layer configurations. Extension to hardware deployment would enable direct validation of the latency claims.

Table 4 summarizes the key similarities and differences between our reproduction and the original HAWQ-V3 implementation.

3.6 Limitations and Threats to Validity

Several limitations of our reproduction merit explicit acknowledgment and constrain the generalizability of our findings.

Simulated Versus True Quantization. Our implemen-

Table 4: Comparison between HAWQ-V3 and our reproduction.

Aspect	HAWQ-V3	Our Work
Dataset	ImageNet	CIFAR-10
Model	ResNet-50	ResNet-18
Sensitivity	Hutchinson trace	Gradient-squared
Optimization	ILP (PULP)	Greedy
Quantization	Integer-only	Simulated
Deployment	TVM on T4 GPU	PyTorch
Runtime	Hours (training)	15 min (total)
Results Comparison		
INT8 accuracy	77.58%	78.88%
Mixed accuracy	76.73%	77.75%
Accuracy drop	0.85 pp	0.95 pp
Size reduction	24%	41%
Speedup	1.23×	Not measured

tation employs simulated quantization, wherein quantized weights are immediately dequantized to floating-point representation prior to convolution operations. This approach accurately models the accuracy impact of quantization noise but does not realize the computational benefits that motivate deployment of quantized models. Integer-only inference with low-precision arithmetic offers substantial speedups on hardware supporting efficient INT4 and INT8 operations, such as NVIDIA Tensor Cores and specialized AI accelerators. Our results validate the accuracy claims of HAWQ-V3 but cannot verify the latency and energy efficiency improvements without hardware deployment.

Sensitivity Estimation Approximation. The gradient-squared approximation to Hessian trace differs from the Hutchinson estimator with Rademacher vectors employed in HAWQ-V3. Our method captures only the diagonal elements of the Hessian and ignores off-diagonal interactions. For layers where off-diagonal elements are significant, the two estimators may produce different sensitivity rankings. However, the diagonal approximation benefits from substantially lower computational cost and memory overhead, which is critical for our resource-constrained setting.

Greedy Versus ILP Optimization. The greedy algorithm is optimal for the two-choice single-constraint setting but does not generalize to scenarios with additional bit-width options (e.g., 2-bit, 6-bit) or multiple simultaneous constraints (e.g., joint size and latency limits). The ILP formulation in HAWQ-V3 handles these extensions naturally and would be necessary for more complex deployment scenarios. However, for the specific problem we address, the greedy solution achieves the global optimum efficiently.

Scale and Generalization. Our experiments employ CIFAR-10 with ResNet-18, which is substantially smaller in scale than ImageNet with ResNet-50. CIFAR-10 images are 32×32 pixels with relatively simple textures and limited intra-class variation, whereas ImageNet images are 224×224 pixels with complex scenes and fine-grained object categories. The sensitivity distribution and optimal bit assignments may differ substantially for larger models and datasets. Direct reproduction on ImageNet would provide stronger evidence of generalization.

Post-Training Versus Quantization-Aware Training. We employ post-training quantization exclusively, wherein a pre-trained FP32 model is quantized without further fine-tuning. Quantization-aware training, wherein the forward pass simulates quantization and gradients flow through a straight-through estimator during training, typically recovers 1 to 2 percentage points of accuracy for aggressive quantization configurations. Our 60% compression target achieves 76.84% accuracy with a 1.86 percentage point drop; quantization-aware training would likely reduce this gap substantially.

Activation Quantization. HAWQ-V3 quantizes both weights and activations, whereas our implementation focuses on weight quantization. Activation quantization introduces additional complexity due to the need for calibration across diverse input distributions and the interaction between batch normalization statistics and activation ranges. Extension to activation quantization would provide a more complete reproduction and potentially enable further compression and speedup.

4 Conclusion

We have successfully reproduced the core methodology of HAWQ-V3 mixed-precision quantization, demonstrating its effectiveness on CIFAR-10 with ResNet-18. Our experiments validate the central claims of the original work and provide insights into the behavior of Hessian-based sensitivity analysis for smaller-scale image classification tasks.

Summary of Contributions. Our reproduction implements three principal components: (i) gradient-based approximation of Hessian trace for per-layer sensitivity estimation; (ii) cost model construction incorporating parameter counts and bit-operations; and (iii) greedy optimization for bit-width assignment under size constraints. These components collectively enable mixed-precision quantization that achieves 41% model size reduction with less than 1 percentage point accuracy degradation on CIFAR-10.

Validation of Key Claims. Our results confirm three central findings from HAWQ-V3. First, sensitivity-based bit allocation is effective and principled. Layers exhibiting higher gradient-squared summation, as a proxy for Hessian trace, benefit from preservation at higher precision, while layers with low sensitivity tolerate aggressive quantization with minimal accuracy impact. Second, mixed-precision quantization achieves superior accuracy versus efficiency trade-offs compared to uniform quantization. Our 40% compression configuration substantially outperforms what uniform 4-bit quantization could achieve without quantization-aware training. Third, optimization for bit-width selection is computationally tractable. Even a simple greedy algorithm produces effective and interpretable bit assignments in under 1 second.

Practical Implications. Our work demonstrates that the HAWQ-V3 methodology generalizes beyond ImageNet-scale tasks to smaller benchmarks commonly employed in resource-

constrained settings. The sensitivity estimation, cost modeling, and optimization components can be implemented efficiently within standard deep learning frameworks without requiring specialized hardware infrastructure. This accessibility facilitates adoption by researchers and practitioners seeking to deploy quantized models on edge devices.

Future Directions. Several extensions would strengthen the reproduction and address current limitations. Implementation of quantization-aware training would likely recover additional accuracy at higher compression levels, particularly for the 60% target configuration where post-training quantization incurs 1.86 percentage points degradation. Deployment using an integer-only framework such as TVM would validate the latency benefits on actual hardware and enable measurement of energy efficiency improvements. Extension to larger models and datasets, particularly ImageNet with ResNet-50, would enable direct quantitative comparison with published HAWQ-V3 results and assess generalization of our findings. Finally, exploration of additional bit-width options (2-bit, 6-bit) and incorporation of activation quantization would provide a more complete picture of the mixed-precision design space.

Concluding Remarks. The success of our reproduction confirms that Hessian-based sensitivity analysis provides a principled and effective foundation for mixed-precision neural network quantization. By identifying layers that are robust to quantization and exploiting this heterogeneity through careful bit allocation, substantial compression can be achieved with minimal accuracy sacrifice. The computational efficiency of the optimization process and the interpretability of the resulting bit assignments make this approach well-suited for practical deployment scenarios.

Acknowledgments

This work was completed as part of ESE 5390: Hardware/Software Co-Design for Machine Learning at the University of Pennsylvania. We thank the course instructors for guidance on project scoping and technical feedback. Computational resources were provided by Kaggle's free GPU tier. We acknowledge the authors of HAWQ-V3 for open-sourcing their methodology, which enabled this reproduction study.

References

[1] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M. W. Mahoney, and K. Keutzer. HAWQ-V3: Dyadic neural network quantization. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.

[2] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer. HAWQ: Hessian aware quantization of neural networks with mixed-precision. In *IEEE/CVF International Conference on Computer Vision*, 2019.

[3] Z. Dong, Z. Yao, D. Arfeen, A. Gholami, M. W. Mahoney, and K. Keutzer. HAWQ-V2: Hessian aware trace-weighted quantization of neural networks. In *Advances in Neural Information Processing Systems*, 2020.

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.

[5] M. van Baalen, C. Louizos, M. Nagel, R. A. Amjad, Y. Wang, T. Blankevoort, and M. Welling. Bayesian bits: Unifying quantization and pruning. *arXiv preprint arXiv:2005.07093*, 2020.

[6] R. Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.

[7] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han. HAQ: Hardware-aware automated quantization with mixed precision. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.

[8] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2016.

[9] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan. PACT: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018.

A Extended Implementation Details

This appendix provides additional technical details regarding our implementation to facilitate reproduction and extension of our work.

A.1 Gradient-Based Sensitivity Computation

Algorithm 1 presents pseudocode for the gradient-based Hessian trace approximation. The procedure accumulates squared gradient norms across K mini-batches to estimate per-layer sensitivity.

Algorithm 1 Gradient-Based Sensitivity Estimation

Input: Model $\mathcal{M}$; data loader $\mathcal{D}$; number of batches K
Output: Sensitivity scores $\{s_\ell\}_{\ell=1}^{L}$
Initialize $s_\ell \leftarrow 0$ for all layers ℓ
for $k = 1$ to K **do**
    Sample mini-batch $(x, y) \sim \mathcal{D}$
    Compute loss $\mathcal{L} \leftarrow \mathrm{CrossEntropy}(\mathcal{M}(x), y)$
    Backpropagate to obtain $\nabla_{W_\ell}\mathcal{L}$ for all layers ℓ
    for each layer ℓ **do**
        $s_\ell \leftarrow s_\ell + \|\nabla_{W_\ell}\mathcal{L}\|_2^2$
    end for
end for
return $\{s_\ell/K\}_{\ell=1}^{L}$

The computational complexity is $O(K \cdot C_{\text{forward}} + K \cdot C_{\text{backward}})$, where C_{forward} and C_{backward} denote the cost of forward and backward passes. For ResNet-18 on CIFAR-10 with $K = 5$, this computation requires approximately 30 seconds on an NVIDIA T4 GPU.

A.2 FLOPs Profiling Implementation

We compute per-layer FLOPs using forward hooks that intercept input and output tensor shapes during inference. For convolutional layers, the FLOPs are computed as:

$$\text{FLOPs}_{\text{conv}} = 2 \cdot C_{\text{out}} \cdot H_{\text{out}} \cdot W_{\text{out}} \cdot \frac{C_{\text{in}} \cdot K_h \cdot K_w}{g}, \quad (8)$$

where g denotes the number of groups (1 for standard convolutions). For fully-connected layers:

$$\text{FLOPs}_{\text{fc}} = 2 \cdot N \cdot d_{\text{in}} \cdot d_{\text{out}}, \quad (9)$$

where N is the batch size. The factor of 2 accounts for both multiplication and accumulation operations in each MAC (multiply-accumulate) instruction.

The BOPS cost model scales FLOPs by the bit-width ratio:

$$\text{BOPS}(b) = \text{FLOPs} \cdot \frac{b}{32}, \quad (10)$$

which assumes that b-bit operations are proportionally cheaper than 32-bit floating-point operations. While this linear scaling is a simplification, it provides a hardware-agnostic proxy for computational cost that correlates with actual latency on specialized accelerators.

A.3 Greedy Algorithm Optimality Proof

For the two-choice single-constraint setting, the greedy algorithm is provably optimal. We provide a brief proof sketch.

Theorem. Let $\mathcal{L} = \{\ell_1, \ldots, \ell_L\}$ be the set of layers sorted by sensitivity $s_1 \leq s_2 \leq \cdots \leq s_L$. Let $b_{\text{high}} = 8$ and $b_{\text{low}} = 4$. Given a size constraint S_{target}, the greedy algorithm that iteratively assigns $\ell_i \leftarrow b_{\text{low}}$ for $i = 1, 2, \ldots$ until $S \leq S_{\text{target}}$ minimizes total sensitivity cost.

Proof Sketch. The problem is equivalent to a fractional knapsack problem with uniform item weights (size reduction $\Delta s_i = p_i \cdot (b_{\text{high}} - b_{\text{low}})$) and values (sensitivity saved s_i). For fractional knapsack, the greedy algorithm that selects items by increasing value-to-weight ratio is optimal. In our setting, all layers have identical per-parameter size reduction, so the value-to-weight ratio is simply the sensitivity s_i. Therefore, quantizing layers in order of increasing sensitivity is optimal.

This optimality guarantee does not hold for more than two bit-width choices or multiple simultaneous constraints, where ILP or other optimization methods are required.

A.4 Hyperparameter Sensitivity

We conducted ablation studies to assess sensitivity to key hyperparameters. Table 5 summarizes results.

Table 5: Ablation study on key hyperparameters for the 40% compression target.

Hyperparameter	Value	Accuracy
Sensitivity batches (K)	3	77.68%
	5 (default)	77.75%
	10	77.77%
Training epochs	3	77.52%
	5 (default)	77.75%
	10	77.81%
Learning rate	0.05	77.58%
	0.1 (default)	77.75%
	0.2	77.63%

The results demonstrate that accuracy is relatively stable across reasonable hyperparameter ranges. Increasing the number of sensitivity estimation batches from 5 to 10 provides marginal improvement (0.02 percentage points), suggesting that 5 batches suffice for stable estimates. Similarly, extending training from 5 to 10 epochs improves baseline accuracy but does not substantially alter the mixed-precision results.